

Asynchroon Javascript



Promises

Async/Await

[example code](#)



Voorbeeld

KdG

Karel de Grote
Hogeschool

ES7

Promise

Async

Await

ES6



Promise

ES5



Callback Hell

Agenda



1. Callbacks
2. Promises
3. Meer Promises...
4. Async/Await

Synchroon
Asynchroon

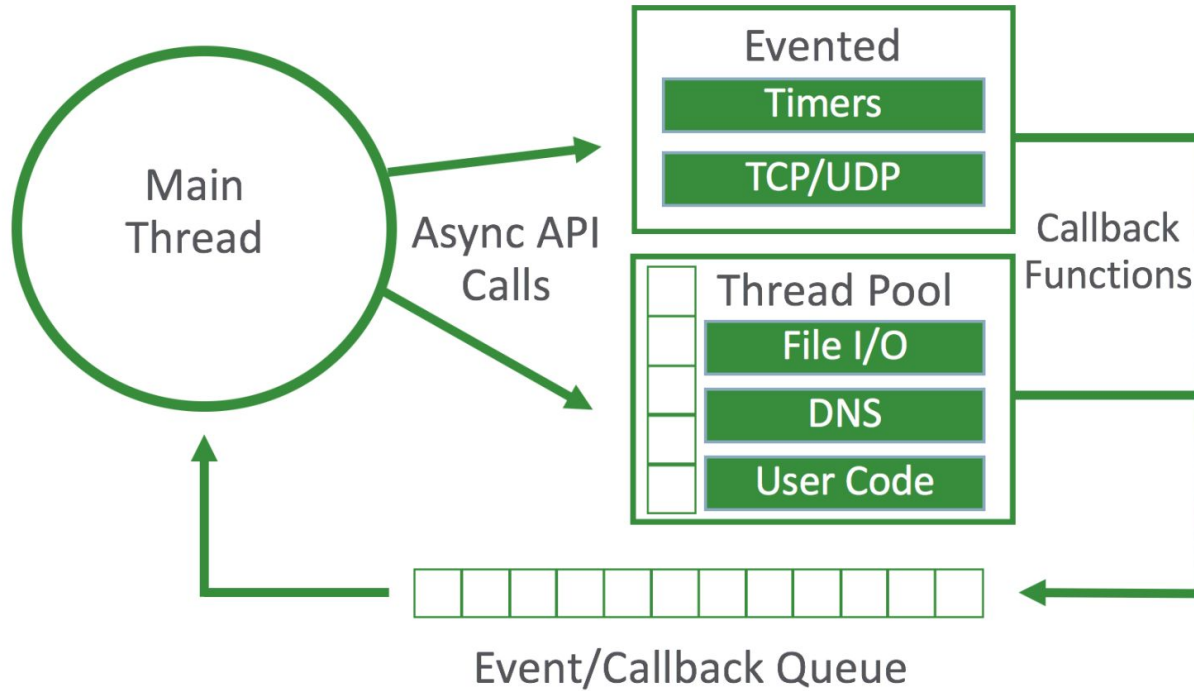
De Person database

- In voorbeelden: databank met array van Person
- We simuleren in de code dat opzoeken in deze databank tot wel één seconde lang duren!

Person.ts

```
const pedigree: Person[] = [  
  {id: 1, born: '1972', name: 'Maud', mother: 2},  
  {id: 2, born: '1945', name: 'Erika', mother: 3},  
  {id: 3, born: '1925', name: 'Goedele', mother: 0},  
  {id: 4, born: '1949', name: 'Electra', mother: 3},  
  {id: 5, born: '1966', name: 'Goedele', mother: 2}  
];
```

Javascript event loop: asynchroon



Javascript 'run-to-completion' principe

Als een JavaScript-functie aangeroepen wordt, zal deze — samen met alle functies die deze synchroon aanroept — **zonder onderbreking** worden uitgevoerd totdat die beëindigd is.

Synchrone code (afgeraden)



dbSearchSync.ts

Voorbeeld

- Javascript werkt met een event loop
- methoden **die lang duren** blokkeren de JavaScript engine: zij worden afgeraden
 - in de dbSearchSync code hebben we de duurtijd van de opzoeking gesimuleerd door met de timer te knoeien
- dbLog drukt de resultaten af met de looptijd

test01DbSearchSync.ts

```
console.info("--- Start ---");  
dbLog(1, sync.dbGetById(1));  
dbLog(2, sync.dbGetById(2));  
dbLog(-1, sync.dbGetById(-1));  
console.info("--- End ---");
```

```
--- Start ---  
  504 ms: {"id":1,"born":"1972","name":"Maud","mother":2}  
  614 ms: {"id":2,"born":"1945","name":"Erika","mother":3}  
 1523 ms: "Invalid ID: -1"  
--- End ---
```

Asynchrone code (OK, maar...)



dbSearchAsync.ts

Voorbeeld

- Asynchrone search methode blokkeert de event loop niet
- Als voorbeeld simuleren een asynchrone call met setTimeout
 - setTimeout eindigt onmiddellijk. De meegegeven methode start na de timeout
 - Extra log in printPerson

repo/dbSearchAsync.ts

```
export function dbGetById( id: number) {  
  let person:Person | undefined;  
  setTimeout(() =>person=printPerson(id),  
    Math.random() * 1000);  
  return person;  
}
```

*return person wordt uitgevoerd
vóór printPerson(id)
dbGetById retournt altijd undefined!*

*printPerson(id) drukt resultaten
lang nadat de searches gedaan zijn
Niet altijd in volgorde van aanroep*

```
--- Start ---  
  0 ms: "Invalid ID: 1"  
  1 ms: "Invalid ID: 2"  
--- End ---  
Log from within async search function!  
  315 ms: {"id":2,"born":"1945","name":"Erika","mother":3}  
Log from within async search function!  
  454 ms: {"id":1,"born":"1972","name":"Maud","mother":2}
```




Callbacks



Callback hell



Callbacks

- JavaScript werkt vaak asynchroon, via
 - Events
 - Callbacks
- We halen het resultaat op in een callback

Callbacks

test02DbSearchCallback.ts

```
import * as callback from "../repo/dbSearchCallback.js"
console.log("--- Start ---");
function printResult(error?: Error , person?: Person) {
  timeLog(error?.message ?? person);
}
callback.dbGetById(1,
  (err, person) => printResult(err, person));
callback.dbGetById(2, printResult);
callback.dbGetById(-1, printResult);
console.log("--- End ---");
```

When a lambda passes all its parameters to one function, you can also use the function reference

```
--- Start ---
--- End ---
```

```
361 ms: {"id":1,"born":"1972","name":"Maud","mother":2}
829 ms: "Person -1 not found"
995 ms: {"id":2,"born":"1945","name":"Erika","mother":3}
```

Asynchroon: resultaten staan niet in volgorde van aanroep

Callbacks

- Callbacks werken goed, maar ingewikkelder voor *sequentiële* logic. *Voorbeeld: Zoek de grootmoeder van een persoon, heeft het resultaat van het opzoeken van een persoon nodig voor een volgende call*

test03DbSearchCallbackNested.ts

```
callback.dbGetById(1, (error, person) => {  
  if (person?.mother) {  
    callback.dbGetById(person.mother, (error, mother) => {  
      if (mother?.mother) {  
        callback.dbGetById(mother.mother, (error, granny) =>  
          printResult(error, {result: "Granny of 1" , person: granny})  
      });  
    }  
  }  
});
```

*Je moet eers de person opzoeken
om de id van de mother te vinden*

*Pyramid of Doom: complexe nesting met diepe
indentering en moeilijke sequentie van sluitende haakjes*

--- Start ---

--- End ---

1393 ms: "The granny of 1 is
{\"id\":3,\"born\":\"1925\", \"name\":
\"Goedele\", \"mother\":0}"

Trump Promise Tracker

Updated Jan. 26, 2018

It's day 398 of the Trump administration with 1,062 days left in his term. We're tracking the progress of 60 pledges he made during his campaign — and whether he achieved his goals.



Promises kept

11

Promises broken

15

Promises

*Hoe leesbare asynchrone
code schrijven?*



Callback

```
callback.dbGetById(1,  
  (err, person) => printResult(err, person));
```

afsluiten dbGetById

Asynchrone afhandeling
in callback functie in
parameterlijst
dbGetById(id, callback)

Promises

```
prom.dbGetById(1)  
  .then(printResult(person))
```

afsluiten dbGetById

toevoegen
.then

Asynchrone afhandeling in
ketting functie na
parameterlijst dbGetById(id)

ES2015 Promises en Node.js

```
import fs from 'node:fs/promises';
```

- promises gebaseerde versie van de fs module.
- `util.promisify` zet een CPS callback functie om in een functie die een Promise terug geeft

```
const readFile = util.promisify(fs.readFile);
```

Promise .then .catch



Voorbeeld

```
test04DBSearchPromise.ts
```

```
prom.dbGetById(2)
  .then(result => dbLog(result.id, result))
  .catch(err => timeLog("Caught " + err.message));
```

- catch is een **kettingfunctie na** then



Promises kept

11

then

Promises broken

15

catch

ES2015 Promises: ketting ipv nesting

- Wat je teruggeeft in then of catch, wordt automatisch in een promise verpakt
 - je kan er dus een nieuwe then/catch op aanroepen
- `a().then(b).then(c); //ketting` -> **fluent style**
 - in plaats van `a(b(c)) //nesting`
 - lange ketting leest makkelijker dan diepe nesting



Promise chaining

ES2015 Promises: volgorde uitvoering

```
a.then(b).then(c);  
console.log("Print direct");  
//... programma gaat verder
```

1. a start
2. Console: Print direct
3. //...programma gaat verder
4. wanneer a gedaan is, then(b)
5. //... programma gaat nog altijd verder
6. wanneer b gedaan is, then(c)

Meerdere opeenvolgende promises



```
test05DBSearchChainedPromises.ts
```

```
prom.dbGetById(1)
  .then(person => {return prom.dbGetById(person.mother) })
  .then(mother => prom.dbGetById(mother.mother) )
  .then(granny => dbLog(granny.id,granny) )
  .catch(err => timeLog("Caught " + err.message));
```

*catch handelt fouten in ALLE
then's af*

*Als de lambda alleen een return bevat,
mag je return en de haakjes weglaten*

*Geen pyramid of doom, maar chaining van
opeenvolgende asynchrone calls*

```
--- Start ---
--- End ---
1017 ms:
{"id":3,"born":"1925","name":"Goedele","mother":0}
```

Voorbeeld: Promise / implementatie



Voorbeeld

repo/dbSearchPromise.ts

```
export default function dbGetById(id: number): Promise<Person> {  
  return timers  
    .setTimeout(Math.random() * 1000, getPerson(id))  
    .then(result =>  
      result ?? Promise.reject(new Error("Reject ID: " + id))  
    )  
}
```

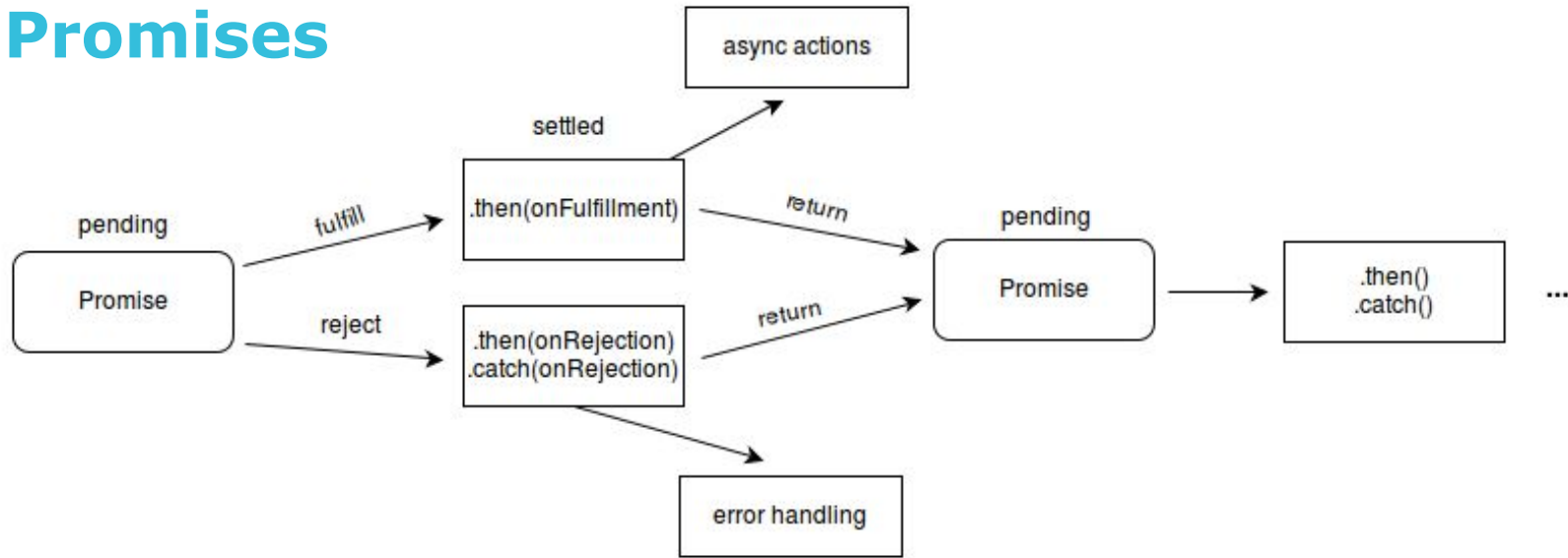
↓ Uitleg in nota's

nullish coalescing operator



Making Promises

Promises



STATE	DESCRIPTION	CALLBACK
pending	De procedure loopt nog, de Promise is nog niet is vervuld.	-
fulfilled	De procedure werd afgerond en was succesvol.	<code>.then()</code>
rejected	De procedure werd afgerond met een fout.	<code>.catch()</code>
settled	De procedure werd afgerond (fout of succesvol), finally callback wordt altijd aangeroepen	<code>.finally()</code>

Promises: meerdere catches

- In het vorige voorbeeld staat de catch op het einde, maar je mag er ook een in het midden van de ketting zetten
 - het is dan meestal de bedoeling de fout te herstellen en verder te gaan in de ketting
 - als je een fout wil doorgeven (naar volgende catch) geef je een rejected Promise terug

```
return Promise.reject(r);
```

- Promise.reject(error) is onmiddellijk in een finale rejected staat
 - (equivalent Promise.resolve(result) is onmiddellijk in een finale resolved staat)



catch (), finally error handling

Meer Promises...

Voorbeeld Asynchrone resultaten verzamelen



Voorbeeld

```
test06DbSearchPromiseArray.ts
```

```
let ra: Person[] = [];
```

```
prom.dbGetById(1)
    .then((result: any) => {
        ra.push(result);
        return prom.dbGetById(2);
    })
    .then((result: any) => {
        ra.push(result);
        return prom.dbGetById(3);
    })
    .then((result: any) => {
        ra.push(result);
        timeLog(ra);
    })
```

```
1758 ms: [{"id":1,"born":"1972","name":"Maud","mother":2},
{"id":2,"born":"1945","name":"Erika","mother":3},
{"id":3,"born":"1925","name":"Goedele","mother":0}]
```

```
.catch((error: any) => timeLog(error));
```


Voorbeeld Asynchrone resultaten verzamelen

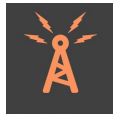
- Een kleine variatie op eerdere voorbeelden
 - Omdat we met promises werken hebben we een then nodig om elk resultaat op te halen en in de array te stoppen
 - In de laatste then, als alle personen gevonden zijn, printen we de array af
 - maar we beginnen pas naar de tweede persoon te zoeken als de eerste gevonden is. Dat is hier niet nodig: om de tweede persoon te zoeken hebben we het resultaat van de eerst gezochte persoon niet nodig.
 - We kunnen alle personen tegelijkertijd zoeken, zolang we maar wachten tot ze allemaal gevonden zijn

Asynchrone resultaten verzamelen

- Deze methoden nemen een **array van Promises** als parameter
- [Promise.all](#): Als **alles** slaagt worden een array van de resultaten aan de then() functie doorgegeven. De volgorde van de resultaten is dezelfde als de volgorde van de functies in de array.
- [Promise.any](#) (ES2020): Geeft resultaat van eerste geslaagde functie terug zodra het beschikbaar is.
- [Promise.allSettled](#) (ES2020): als Promise.all maar geeft ook resultaat van niet geslaagde promises terug. Resultaat is een object met een van volgende vormen:

```
{status:"fulfilled", value:result}
```

```
{status:"rejected", reason:error}
```



[Parallel uitvoeren van promises:](#)

Asynchrone resultaten verzamelen:



```
test07DbSearchPromiseAll.ts
```

```
Promise.all([prom.dbGetById(1), prom.dbGetById(2),  
  prom.dbGetById(3)])  
  .then(timeLog)  
  .catch(timeLog);
```

array van Promises

Parallele uitvoering. Als de traagste promise gedaan is krijg je het resultaat (na 850 ms!) Resultaten blijven in volgorde van opgave, niet in de volgorde dat ze geëindigd zijn!

850 ms:

```
[{"id":1,"born":"1972","name":"Maud","mother":2},  
 {"id":2,"born":"1945","name":"Erika","mother":3},  
 {"id":3,"born":"1925","name":"Goedele","mother":0}]
```

Asynchrone resultaten verzamelen:



```
test08DbSearchPromiseAny.ts
```

```
Promise.any([prom.dbGetById(1), prom.dbGetById(2),  
  prom.dbGetById(3)])  
  .then(timeLog)  
  .catch(timeLog);
```

1 resultaat (de snelste)!

287 ms:

```
{"id":1,"born":"1972","name":"Maud","mother":2}
```

Asynchrone resultaten verzamelen:



```
test09DbSearchPromiseAllSettled.ts
```

```
Promise.allSettled([prom.dbGetById(1),  
  prom.dbGetById(2),  
  prom.dbGetById(-1)])  
  .then(timeLog)  
  .catch(timeLog);
```

*extra status property in
returnwaarde*

```
718 ms:[{"status":"fulfilled","value":{"id":1,"born":"1972","name":"Maud","mother":2}},  
  {"status":"fulfilled","value":{"id":2,"born":"1945","name":"Erika","mother":3}},  
  {"status":"rejected","reason":{}}  
]
```

Async Await



[async/await](#)

Promise

```
function searchPromise() {  
  prom.dbGetById(1)  
    .then(p => {  
      timeLog({type:"prom",person: p});  
      return prom.dbGetById(p.mother)  
    })  
    .then(p => {  
      timeLog({type:"prom",person: p});  
      return prom.dbGetById(p.mother)  
    })  
    .then(p =>  
      timeLog({type:"prom",person: p}));  
}
```

Synchroon

```
function searchSync() {  
  
  let p: Person = sync.dbGetById(1);  
  timeLog({type:"sync",person: p});  
  
  p = sync.dbGetById(p.mother);  
  timeLog({type:"sync",person: p});  
  
  p = sync.dbGetById(p.mother);  
  timeLog({type:"sync",person: p});  
}
```

*We vergelijken searcher code (links), die met Promises werkt,
met searcherSync (rechts) (gewone synchrone code) die telkens wacht
en een resultaat teruggeeft.*

test10DbSearchAwait.ts

Eenvoudige asynchrone code

- Code die je schrijft met Promises is eenvoudiger dan code met callback handlers
 - geen complexe nesting
 - ligt dicht bij structuur van synchrone code die ontwikkelaars gewoon zijn
- Maar synchrone code is natuurlijk nog eenvoudiger
- Async/await is een nieuwe syntax om te werken met promises, maar met een syntax die nog dicht bij synchrone code ligt

*Hoe leesbare
asynchrone code
schrijven?*



Intermezzo: Generators

Een generator is een technisch hoogstandje van ES6 te herkennen aan **function*** hier enkel ter illustratie. Valt buiten de leerstof!

```
function stepOverGenerator() {  
  let g = sumGenerator();  
  let step1 = g.next();  
  console.log(`step1: ${JSON.stringify(step1)}`);  
  let step2 = g.next(1);  
  console.log(`step2: ${JSON.stringify(step2)}`);  
  let step3 = g.next(2);  
  console.log(`step3: ${JSON.stringify(step3)}`);  
}
```

```
function* sumGenerator() {  
  let sum = 0;  
  let x = yield sum;  
  sum += x;  
  console.log(`+${x} -> ${sum}`);  
  let y = yield sum;  
  sum += y;  
  console.log(`+${y} -> ${sum}`);  
  return `totaal: ${sum}`;  
}
```

```
$ node stepOverGenerator.js  
step1: {"value":0,"done":false}  
+1 -> 1  
step2: {"value":1,"done":false}  
+2 -> 3  
step3: {"value":"totaal: 3","done":true}
```

De code in de generator **LIJKT op serieel** uitvoerbare code maar in werkelijkheid is het een opeenvolging van callbacks die van buitenaf kunnen gestuurd worden. Let ook op de uitwisseling van variabelen!

Zou je hiermee de promises callbacks ook kunnen omzetten naar een sequentie?

Async/Await

```
async function searchAsync() {  
  let p = await prom.dbGetById(1);  
  timeLog({type:"await",person: p});  
  p = await prom.dbGetById(p.mother);  
  timeLog({type:"await",person: p});  
  p = await prom.dbGetById(p.mother);  
  timeLog({type:"await",person: p});  
}
```

await een Promise in plaats van *.then(...)*
enkel in **async** function

await laat werken met Promises (nog) meer op gewone synchrone code lijken
maar het is het NIET!

Synchroon

```
function searchSync() {  
  let p: Person = sync.dbGetById(1);  
  timeLog({type:"sync",person: p});  
  p = sync.dbGetById(p.mother);  
  timeLog({type:"sync",person: p});  
  p = sync.dbGetById(p.mother);  
  timeLog({type:"sync",person: p});  
}
```

pedigree/test10DbSearchAwait.ts



Voorbeeld

await

- await voor asynchrone aanroepen laat die er net zo uitzien als gewone code
 - Je hebt geen .then ketting functie nodig om het resultaat in op te vangen, maar kan het behandelen als een gewone return waarde
- Als je await in een functie gebruikt moet de functie voorafgegaan worden door `async`
- Deze functies zijn asynchroon (eindigen direct) en geven een promise terug
- Je kan async await combineren met Promise.all, Promise.any...
- Top level await (ES2022) laat toe await buiten een functie te gebruiken in een module. (Node.js v16, recente browsers)

```
export const item1 = await dbGetById(1);
```

await

- await werkt natuurlijker met constructies als try/catch, loops, if/else...
- Toch zijn er soms subtiele verschillen, denk er steeds aan dat
 - er eigenlijk een onderliggende Promise.then() is (!)
 - async functies die je zonder await aanroept
 - worden parallel uitgevoerd worden
 - geven een Promise terug
 - eindigen onmiddellijk, de code gaat onmiddellijk verder met de volgende instructie

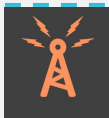
Promises: .then .catch

test10DbSearchAwait.ts



Voorbeeld

```
function searchError() {  
  prom.dbGetById(1)  
    .then(p => {  
    timeLog({type:"then",person:p});  
    return prom.dbGetById(p.id - 10)  
  })  
    .then(p => {  
    timeLog({type:"then",person:p});  
    return prom.dbGetById(2)  
  })  
    .then(p=> ({type:"then",person:p}))  
    .catch(r => timeLog("catch " + r));  
}
```



catch (), finally error handling

```
527 ms: {"type":"then"  
  , "person":{"id":1,"born":"1972","name":"Maud","mother":2}}  
1360 ms: "catch Error: Reject ID: -9"
```

Async/await: try catch

```
async function searchErrorAwait() {  
  try {  
    let p = await prom.dbGetById(1);  
    timeLog({type:"try",person:p});  
    p = await prom.dbGetById(p.id - 10);  
    timeLog({type:"try",person:p});  
    p = await prom.dbGetById(2);  
    timeLog({type:"try",person:p});  
  } catch (r) {  
    timeLog("try catch " + r);  
  }  
}
```

```
10 ms: {"type":"try",  
  "person":{"id":1,"born":"1972","name":"Maud","mother":2}}  
477 ms: "try catch Error: Reject ID: -9"
```

Samenvatting



1. Callbacks
2. Promises
3. Meer Promises...
4. Async/Await



- [https://javascript.info/Asynchroon programmeren](https://javascript.info/Asynchroon_programmeren)
- [recommended reading: javascript.info promisification](#)
- <https://developers.google.com/web/fundamentals/promisers/promises>